

ليه دايما بشوف ان فيه lifecycle في الاندرويد واضحه عن
jetpack compose

السؤال ده مهم جدًا، وفعلًا ناس كثير بتلاحظ إن ال lifecycle في الأندرويد التقليدي أوضح بكثير من
Jetpack Compose — وده مش صدفة 🙌

📌 أولًا: ليه ال lifecycle واضح في النظام القديم (Views)؟

في النظام القديم (XML + Activities/Fragments):

• عندك lifecycle واضح وصريح زي:

- onCreate()
- onStart()
- onResume()
- onPause()
- onDestroy()

♦ ده لأن UI مرتبط مباشرة بـ:

- Activity
- Fragment

يعني كل حاجة ليها "حياة" واضحة بتبدأ وتنتهي.

📌 طب ليه في Compose مش باين؟

في Jetpack Compose الوضع مختلف تمامًا:

💡 Compose مبني على فكرة:

UI = Function

يعني:

```
@Composable
fun MyScreen() {
    Text("Hello")
}
```



Kotlin 1.8

! مفيش lifecycle تقليدي هنا، لأن:

- الفنكشن دي ممكن تتنفذ كذا مرة (Recomposition)
- ومفيش "start" أو "destroy" واضح زي Activity

الفرق الجوهرى

Compose	النظام القديم
Lifecycle ديناميكي	Lifecycle ثابت
UI بيتعاد بناؤه كتير	UI بيتبني مرة
implicit (غير مباشر)	واضح (callbacks)

طيب Compose مفيهوش lifecycle خالص؟ 🤔

لا، فيه... بس بشكل مختلف 🙋

Compose بيعتمد على:

Recomposition 1

- كل ما state يتغير → UI يعيد الرسم
- ده يعتبر "حياة" جديدة للـ UI

Side Effects APIs 2

زي:

- LaunchedEffect
- DisposableEffect
- remember

دي بتديك lifecycle شبه القديم:

مثال:

```
LaunchedEffect(Unit) {
    // زي onCreate أو onStart
```



Kotlin </>

```
DisposableEffect(Unit) {
    onDispose {
        // زي onDestroy
    }
}
```

3 ربطه بـ ViewModel lifecycle

Compose يعتمد على:

- ViewModel

وده لسه عنده lifecycle واضح:

- بيعيش مع ال Activity أو Fragment

🔥 الخلاصة

انت شايف lifecycle أوضح في النظام القديم لأن:

- هو Explicit (مكتوب قدامك)

لكن في Compose:

- هو Implicit (مستخبي جوه النظام + state + recomposition)

🎯 تشبيه بسيط

- النظام القديم = فيلم له بداية ونهاية واضحة 🎬
- Compose = Stream لايف بيتحدث طول الوقت ●

لو حابب أشرحلك:

- امتى تستخدم remember VS LaunchedEffect
- أو أدبك مقارنة عملية بين XML و Compose في نفس المثال

قولّي 👍



طيب اشرح الحاجات دي بتديك lifecycle شبه القديم:

تمام، خلينا نفهم بعمق إزاي الحاجات دي في Jetpack Compose بتديك lifecycle شبه القديم 📌

الفكرة الأساسية 🎯

في النظام القديم (Activity/Fragment) كان عندك:

- onCreate
- onStart
- onDestroy

لكن في Compose:

مفيش lifecycle جاهز... انت اللي "بتكوّنه" باستخدام APIs زي LaunchedEffect و DisposableEffect

onCreate / onStart → LaunchedEffect 1 🔥

بيعمل ايه؟ 📌

- بيتنفذ أول ما ال Composable يظهر
- ويتنفذ ثاني لو ال key اتغير

مثال: ✅



Kotlin </>

```
@Composable
fun MyScreen(userId: Int) {

    LaunchedEffect(userId) {
        // يتنفذ أول مرة
        // اتغير userId ويتنفذ ثاني لو
        fetchUserData(userId)
    }

}
```

يشبه: 💡

- onCreate () → أول تشغيل
- onStart () → لما البيانات تتغير

- listener
- sensor
- broadcast receiver

remember → حفظ الحالة (مش lifecycle مباشر) 3 🔥

ييعمل ايه؟ 📌

- بيخزن قيمة طول ما ال Composable عايش

مثال: ✅



Kotlin </>

```
@Composable
fun Counter() {
    val count = remember { mutableStateOf(0) }

    Button(onClick = { count.value++ }) {
        Text("${count.value}")
    }
}
```

يشبه: 💡

- variable محفوظ جوه Activity

لكن:

- ! مش بيتعامل مع start/destroy
- بس بيحافظ على state أثناء ال recomposition

rememberUpdatedState (مستوى أعلى شوية) 4 🔥

ليه؟ 📌

عشان تحافظ على آخر قيمة جوه effect بدون ما تعيد تشغيله

ربط Compose بـ lifecycle الحقيقي 5 🔥

لو عايز lifecycle حقيقي زي القديم:



Kotlin </>

```
val lifecycleOwner = LocalLifecycleOwner.current
```

وتشتغل مع:

Lifecycle •

مقارنة مباشرة (مهمة جدًا)

Compose	القديم
LaunchedEffect(Unit)	onCreate
LaunchedEffect(key)	onStart
DisposableEffect → onDispose	onDestroy
remember	variables

الخلاصة

Compose مفهوش lifecycle صريح
لكن بيديك أدوات تبني بيها lifecycle بنفسك:

- LaunchedEffect → تشغيل كود
- DisposableEffect → تنظيف
- remember → حفظ state

أهم نقطة تفهمها

lifecycle في Compose = مبني على state + recomposition
مش على callbacks زي زمان

لو حابب أدريك:

- مثال (API call + loading + cleanup) real
- أو أوريك bug بيحصل لو استخدمتهم غلط 🤔

قولّي 👍